Ron Hensley

ronkhensley@gmail.com

AAS in AI Software Engineering

PY102

What makes an algorithm good?

Today
Welcome to your next lesson: What makes an algorithm good?

I'm preparing your personalized lesson based on what you've learned, what's coming up next, and the best way to help it all stick.

Step by step, you’re moving forward.

Imagine you’re a mail carrier 📨

You have a list of houses and a plan (an algorithm) for how to deliver mail:

You follow your plan step by step.
The question is: is this a good plan or a bad one?

For any algorithm, we usually care about three things:

Does it always give the right result?
How long does it take / how much work does it do?
Is the code easy for humans to read and fix later?

We’ll dig into each of these slowly, with tiny bits of Python and some step-counting.

To start simple:

When you hear “good algorithm”, which of these feels most important first?

It always gives the right answer, even if it’s slow
It is very fast, but sometimes gives the wrong answer

You don’t need to justify it—just tell me which one you’d pick and we’ll build from there.

Always gives the right answer

Nice instinct 💯 — if an algorithm gives the wrong answer, it doesn’t matter how fast or pretty it is.

This property has a name:

Correctness = the algorithm always gives the right answer for every valid input.

Example:

In Python, say we want the sum of a list:

Python

```
def sum_list(nums):
    total = 0
    for n in nums:
        total = total + n
    return total
```

If this function gives the right sum for:

[1, 2, 3]
[]
[10, -5, 7]
any other list of numbers

…then we say the algorithm is correct.

Quick check before we move on:

If an algorithm is very fast but sometimes returns the wrong result, would you call it correct? Just answer yes or no.

NO

Exactly 👍 — if it sometimes gives the wrong result, it's not correct.

So we've locked in idea #1:

A good algorithm must be correct. No exceptions.

Next idea: even among correct algorithms, some are "better" because they do less work.

Tiny example with mail delivery:

Plan A: walk straight down the street once, delivering to each house in order.
Plan B: for each house, you keep walking back to the post office and then out again 😅
Both will eventually deliver all the mail (so both are correct), but one clearly does a lot more walking.

In algorithms, we call this idea:

Efficiency = how much work (how many steps) the algorithm does.

Make sense so far:

Correctness = always right
Efficiency = how much work it does

Just answer: should we dive into efficiency next, using a small Python example? (yes/no)

Yes, let’s explore efficiency

Awesome, let’s make efficiency very concrete ❁

Think of efficiency as:

How many basic steps does this algorithm do to finish its job?

Let’s take a simple Python function that adds numbers in a list:

Python

```
def sum_list(nums):
    total = 0          # 1 step (set total)
    for n in nums:     # loop setup
        total += n     # 1 step each time through the loop
    return total       # 1 step (return)
```

If nums = [5, 8, 2], then:

total = 0 → 1 step
Loop runs 3 times → 3 steps (total += n each time)
return total → 1 step

So we can roughly say: about 1 + (number of items) + 1 steps.

I want to check your feel for this idea:

If the list has 6 numbers instead of 3, what changes in this function — the code, or just the number of times the loop runs?

You can answer in your own words, or just: “loop runs more times”.

8 steps 1 + 6 + 1

Nice, you’re thinking in exactly the right way 👍

For 6 numbers:

total = 0 → 1 step
Loop runs 6 times → 6 steps
return total → 1 step
Total ≈ 8 steps ☑

So you’ve just done a tiny bit of efficiency analysis:

Same code
Bigger input list
More loop steps
Next, I want to show you that two different correct algorithms can do very different amounts of work for the same task.

Imagine two ways to sum a list:

one passes through the list once
another does something wasteful and loops inside a loop

We’ll count steps and compare.

Before I show that code: do you feel okay counting steps like you just did, if I give you another small function? (yes/no is fine)

im comfy ith it

Love it 😄 That's exactly the muscle we need.

Let's compare two correct algorithms that both sum a list but do different amounts of work.

Algorithm A (what you already saw):

Python

```
def sum_list_once(nums):
    total = 0
    for n in nums:
        total += n
    return total
```

Now imagine a worse but still correct version:

Python

```
def sum_list_slow(nums):
    total = 0
    for i in range(len(nums)):
        # inner loop adds nums[i] one step at a time
        step = 0
        while step < nums[i]:
            total += 1
            step += 1
    return total
```

Both give the same answer for positive integers:

they're both correct
but the second one clearly does way more work 😳
To keep it simple, let's just focus on Algorithm A for a moment.

For nums = [3, 4, 5], you already know how to think about steps:

total = 0 → 1 step
loop body runs 3 times → 3 steps
return total → 1 step

Total ≈ 5 steps

Make sense so far that:

both algorithms are correct
but they don't do the same amount of work?

If yes, next I'll ask you to compare their work on a tiny concrete example (I'll keep the numbers very small).

yes

Great 👍 let's compare them on a very small, concrete case.

We'll use this list:

Python

nums = [2, 1]
Algorithm A:

Python

```
def sum_list_once(nums):
    total = 0
    for n in nums:
        total += n
    return total
```

Step counting for nums = [2, 1]:

total = 0 → 1 step
Loop over nums:
first time (n = 2): total += n → 1 step
second time (n = 1): total += n → 1 step
return total → 1 step

So Algorithm A does about 4 steps here.

I'll pause on Algorithm B for a moment.

Key idea I want you to hold:

Both algorithms give the same sum → both correct
Algorithm B will do more steps, especially as numbers get bigger

Does this core idea land: "Several correct algorithms can solve the same problem, but some do way more work than others"?

You can answer "yes" or "not really yet".

Yes, that makes sense

What do you think? 👇

Which quality of a good algorithm is about always giving the right answer for every valid input?

A.

Correctness

B.

Efficiency

C.

Readability

Nice work, Ron 🎉 You nailed it.

Correctness is exactly the quality that's about always giving the right answer for every valid input.

If an algorithm is fast and pretty but sometimes wrong, it's not a good algorithm. Correctness comes first; efficiency and readability matter after we're sure the answers are always right.

Quick check-in before we move on:

Would you like to:

look a bit more at efficiency with another tiny step-count example, or
talk about readability next (how to make algorithms easy for humans to understand)?
Talk about readability

Nice choice 😄 Readability is what future-you will thank you for.

Simple idea:

Readability = how easy it is for a human to read, understand, and change the code.

Two algorithms can do the exact same thing and be equally correct and efficient, but one is a mess to read and the other is clear.

Look at these two Python snippets that both sum a list:

Messy version:

Python

```
def f(a):
    s = 0
    for x in a:
        s += x
    return s
```

Clearer version:

Python

```
def sum_list(nums):
    total = 0
    for num in nums:
        total += num
    return total
```

They do the same work, but:

names like sum_list, nums, total tell a story
you can guess what it does without much thinking

That’s readability in action.

Quick question for you:

Between those two versions, which would you rather maintain 6 months from now when you forgot the details: the first one or the second one? Just say “first” or “second”.

second

Nice call 👍 I’d absolutely pick the second one too.

So now you’ve seen all three qualities of a “good” algorithm:

Correctness – always gives the right answer
Efficiency – does the work in a reasonable number of steps
Readability – humans can easily read, understand, and change it later

They all matter:

An algorithm that’s correct but unreadable is painful to maintain.
An algorithm that’s readable but wrong is useless.
An algorithm that’s fast but wrong is dangerous.
To lock this in, let’s do one more tiny exercise about efficiency + step-counting.

Imagine this function in Python:

Python

```
def count_items(nums):
    count = 0
    for n in nums:
        count += 1
    return count
```

Question:

If nums = [4, 7, 1, 9] (4 items), how many times does the line count += 1 run?

You can just answer with a number.

4

Exactly ☑

For nums = [4, 7, 1, 9]:

The loop runs once per item → 4 times
So count += 1 runs 4 times

You just:

looked at a loop
connected “how many items” → “how many times the body runs”

That’s the core of thinking about efficiency in terms of steps. 🎯

Let’s push that idea one tiny step further, still with the same function:

Python

```
def count_items(nums):
    count = 0
    for n in nums:
        count += 1
    return count
```

If the list has:

3 items → count += 1 runs 3 times
4 items → runs 4 times
6 items → runs 6 times

So as the input size grows, the number of loop steps grows with it.

I want to check this pattern with you:

Imagine nums has 10 items.

How many times does count += 1 run? Just the number is fine.

10

Perfect ☑

You've just shown you can:

spot correctness (always right answer),
reason about efficiency by counting steps as input grows,
and recognize readability as clear, human-friendly code.

That hits all the goals for this lesson 👌

Before we consider wrapping this lesson up, is there any part you'd like to see one more quick example of:

correctness,
efficiency / step-counting,
or readability?

You can pick one of those, or say "I'm good for this lesson."

I'm good for this lesson

Nice work today, Ron 🎯

Here's what you're taking away from this lesson:

A good algorithm has three key qualities:

Correctness – it always gives the right answer for every valid input.
Efficiency – it doesn't do more work (steps) than it needs to.
Readability – humans can easily read, understand, and change the code.
You saw that:

Two different algorithms can both be correct, but one can do way more steps than the other.
By counting loop steps (like how many times count += 1 runs), you can see how work grows as the input list gets bigger.
You handled the step-counting and the concepts really smoothly 👌

If you're ready, you can jump into the next lesson in the Learn tab, and we'll keep building on this.
If something from today pops back into your mind and feels fuzzy later, just tell me what it is and we'll revisit it.